efficient JavaScript under the hood. During this process, source maps are also generated, linking the transpiled code back to the original JSX. This allows browser developer tools to display the original JSX during debugging, significantly improving the developer experience and maintainability of the codebase.

Follow-up question (fundamental skill): "How does JSX handle children props?"

Answer: JSX automatically passes any content between a component's opening and closing tags as `props.children`. For example, in `<Greeting>Hello</Greeting>`, the text "Hello" is passed as `props.children` to the `Greeting` component. This feature simplifies component composition and enables the creation of wrapper components capable of rendering arbitrary child content.

Follow-up question (applied skill): "How can dynamic children composition be handled in JSX?"

Answer: Dynamic children composition allows developers to pass components or functions as children and control how they are rendered. This pattern is particularly powerful when building flexible, reusable UI libraries. Listing 4.1 demonstrates the function-as-children pattern, where a parent component provides dynamic data to its child.

Listing 4.1 A parent component provides dynamic data to its child

```
const DataProvider = ({
  children,
}: {
  children: (data: string) => React.ReactNode;
}) => {
  const data = "Dynamic data from parent";
  return <>{children(data)}</>; #A
};
<DataProvider>
  {(data) => <p>{data}</p>}
</DataProvider>;
```

Probing question (advanced): "How do JSX transpilation and advanced